



finis
Universidad Finis Terrae
Ing. Civil en Informática y Telecomunicaciones

Universidad Finis Terrae
Facultad de Ingeniería
Estructura de datos y Algoritmo
Sección: (2621478)
Fecha: Junio 2026

Proyecto estructura de datos y algoritmo: Búsqueda de Patrones en Texto mediante Fuerza Bruta e Indexación con Tablas Hash

Alumnos:

Ricardo Steelheart
Francisco Ossa

Profesor(a) y Ayudante:

Rodrigo Paredes Moraleda
Eduardo Velasquez Gil



Índice

1. Introducción	3
1.1. Objetivo general	3
1.2. Objetivos específicos	3
2. Análisis del Problema	3
2.1. Descripción del problema	3
2.2. Supuestos	4
2.3. Casos de borde	4
3. Solución del Problema	4
3.1. Algoritmo de Fuerza Bruta	4
3.1.1. Complejidad	4
3.2. Algoritmo Indexado	5
3.2.1. Complejidad	5
3.2.2. Estructura del índice	5
3.2.3. Complejidad	5
4. Diagrama de Estados	7
5. Implementación	7
5.1. Pseudocódigo de Fuerza Bruta	7
5.2. Pseudocódigo del Índice Hash	8
5.3. Cliente Interactivo	8
6. Modo de Uso	9
6.1. Ejecución	9
6.2. Ejemplo de ejecución	9
7. Pruebas Experimentales	10
7.1. Texto de Prueba 1	10
7.2. Texto de Prueba 2	10
7.3. Metodología	11
8. Resultados	12
8.1. Resultados del Texto 1	12
8.2. Resultados del Texto 2	13
8.3. Gráficos	14
9. Análisis de Resultados	14
10. Conclusiones	14
11. Anexos	15

Introducción

En la actualidad una gran cantidad de información es almacenada en archivos y sistemas informáticos. De este modo se vuelve importante contar con métodos que permitan encontrar datos específicos de forma rápida y eficiente.

Un método mediante el que se puede buscar información es por fuerza bruta, la cual revisa todas las líneas del archivo hasta encontrar el dato solicitado. Este método es sencillo, pero puede tardar más tiempo cuando la cantidad de información es grande.

Otra alternativa es utilizar indexación, estructurando previamente la información para facilitar las búsquedas. En este proyecto experimental se utilizaron dos tipos de índices: un diccionario de Python y una tabla hash propia. Estas estructuras permiten encontrar información más rápidamente, ya que evitan recorrer todo el archivo en cada consulta.

Durante este trabajo se implementaron y compararon estos tres métodos de búsqueda con el objetivo de analizar sus tiempos de ejecución y determinar cuál ofrece un mejor rendimiento.

Objetivo general

- Comprobar mediante pruebas y mediciones de tiempo que las búsquedas indexadas son más eficientes que la búsqueda secuencial al trabajar con archivos de texto.

Objetivos específicos

- Implementar un método de búsqueda secuencial o fuerza bruta.
- Implementar métodos de búsqueda indexada utilizando un diccionario de Python y una tabla hash propia.
- Medir los tiempos de ejecución de cada método de búsqueda.
- Comparar los resultados obtenidos para analizar la eficiencia de cada alternativa.

Análisis del Problema

Descripción del problema

El problema a resolver consiste en buscar una palabra o cadena de caracteres (patrón) dentro de un archivo de texto plano correspondiente a la obra *El Hombre Bicentenario* de Isaac Asimov y de *El Principito* como prueba. Para solucionar esto se desarrolló un programa que implementa diferentes métodos de búsqueda entre los cuales se encuentran: búsqueda secuencial por fuerza bruta, la búsqueda indexada propia de Python y búsqueda indexada utilizando una tabla hash propia.

El usuario podrá ingresar cualquier palabra no vacía como patrón de búsqueda. El programa procesa dicha entrada y realiza la búsqueda utilizando el método que sea seleccionado por el usuario.

Una vez finalizada la búsqueda, el sistema devuelve el listado con los números de línea en los cuales aparece el patrón buscado. Además, el programa permite crear índices,

ejecutar consultas desde un archivo y medir los tiempos de ejecución de cada método con el fin de comparar su eficiencia.

Supuestos

- El archivo de texto se encuentra en formato UTF-8.
- El archivo puede contener cualquier cantidad de líneas.
- Una palabra puede aparecer varias veces dentro del texto.
- Las palabras se convierten a minúsculas antes de realizar las búsquedas.
- El usuario debe ingresar una palabra o patrón no vacío para realizar la búsqueda.

Casos de borde

- El archivo de texto está vacío.
- La palabra buscada no existe en el archivo.
- La palabra buscada aparece varias veces en distintas líneas.
- El usuario ingresa una palabra vacía.
- Se intenta buscar sin haber cargado previamente un archivo.
- Se intenta utilizar un índice que aún no ha sido creado.

Solución del Problema

Algoritmo de Fuerza Bruta

La búsqueda por fuerza bruta consiste en revisar todas las líneas del archivo una por una hasta encontrar la palabra que ingresó el usuario. Para cada línea, el programa verifica si el patrón buscado está presente. Si lo encuentra, guarda el número de la línea donde aparece.

Al finalizar el recorrido del archivo, se muestra una lista con todas las líneas donde se encontró la palabra buscada. Este método es fácil de implementar, pero puede ser lento cuando el archivo contiene mucha información, ya que debe revisar todo el texto cada vez que se realiza una búsqueda.

3.1.1. Complejidad

[$O(n)$]

donde (n) corresponde a la cantidad de líneas que debe revisar el programa.

Algoritmo Indexado

La búsqueda indexada funciona creando previamente un índice con las palabras que aparecen en el texto y las líneas donde se encuentran. De esta forma, cuando el usuario realiza una búsqueda, el programa no necesita recorrer todo el archivo nuevamente.

En este trabajo se utilizaron dos tipos de índices: un diccionario de Python y una tabla hash propia. Para construir el índice, el programa recorre todas las líneas del archivo, separa las palabras y guarda cada una junto con el número de línea donde aparece.

En la tabla hash, cada palabra pasa por una función hash que calcula una posición dentro de la tabla. Luego la palabra se almacena en esa posición junto con sus líneas de aparición.

Gracias a este índice, las búsquedas son más rápidas porque el programa puede acceder directamente a la información sin revisar todo el archivo.

3.2.1. Complejidad

[$O(1)$]

En promedio, la búsqueda en una tabla hash tiene tiempo constante, ya que accede directamente a la posición donde se encuentra la palabra.

En algunos casos, cuando varias palabras quedan almacenadas en la misma posición, la complejidad puede aumentar hasta:

[$O(n)$]

donde (n) representa la cantidad de elementos almacenados en esa posición.

3.2.2. Estructura del índice

El índice almacena cada palabra encontrada en el texto junto con una lista de las líneas donde aparece. De esta forma, cuando se realiza una búsqueda, el programa puede obtener directamente las líneas asociadas a la palabra sin recorrer nuevamente todo el archivo.

```
===== MENU PRINCIPAL =====
1. Cargar archivo de texto
2. Buscar por fuerza bruta
3. Crear índice con diccionario de Python
4. Crear índice con tabla hash propia
5. Buscar usando diccionario de Python
6. Buscar usando tabla hash propia
7. Ejecutar consultas desde archivo
8. Medir tiempos
0. Salir
Seleccione una opción: 6
Ingrese patrón a buscar: robot
Líneas encontradas: [42, 44, 46, 57, 62, 72, 74, 75, 93, 95, 96, 153, 163, 186, 203, 213, 216,
5, 593, 595, 597, 599, 602, 603, 604, 626, 628, 658, 708, 716, 721, 726, 734, 735, 739, 740, 7
```

Figura 1: Ejemplo de índice generado a partir del texto

3.2.3. Complejidad

La construcción del índice requiere recorrer todo el archivo una vez para procesar sus palabras, por lo que su complejidad es:



[$O(n)$]

donde (n) corresponde a la cantidad de palabras procesadas.

Una vez creado el índice, las búsquedas son mucho más rápidas, ya que se puede acceder directamente a la información almacenada.

[$O(1)$]

en promedio para cada consulta.

Diagrama de Estados

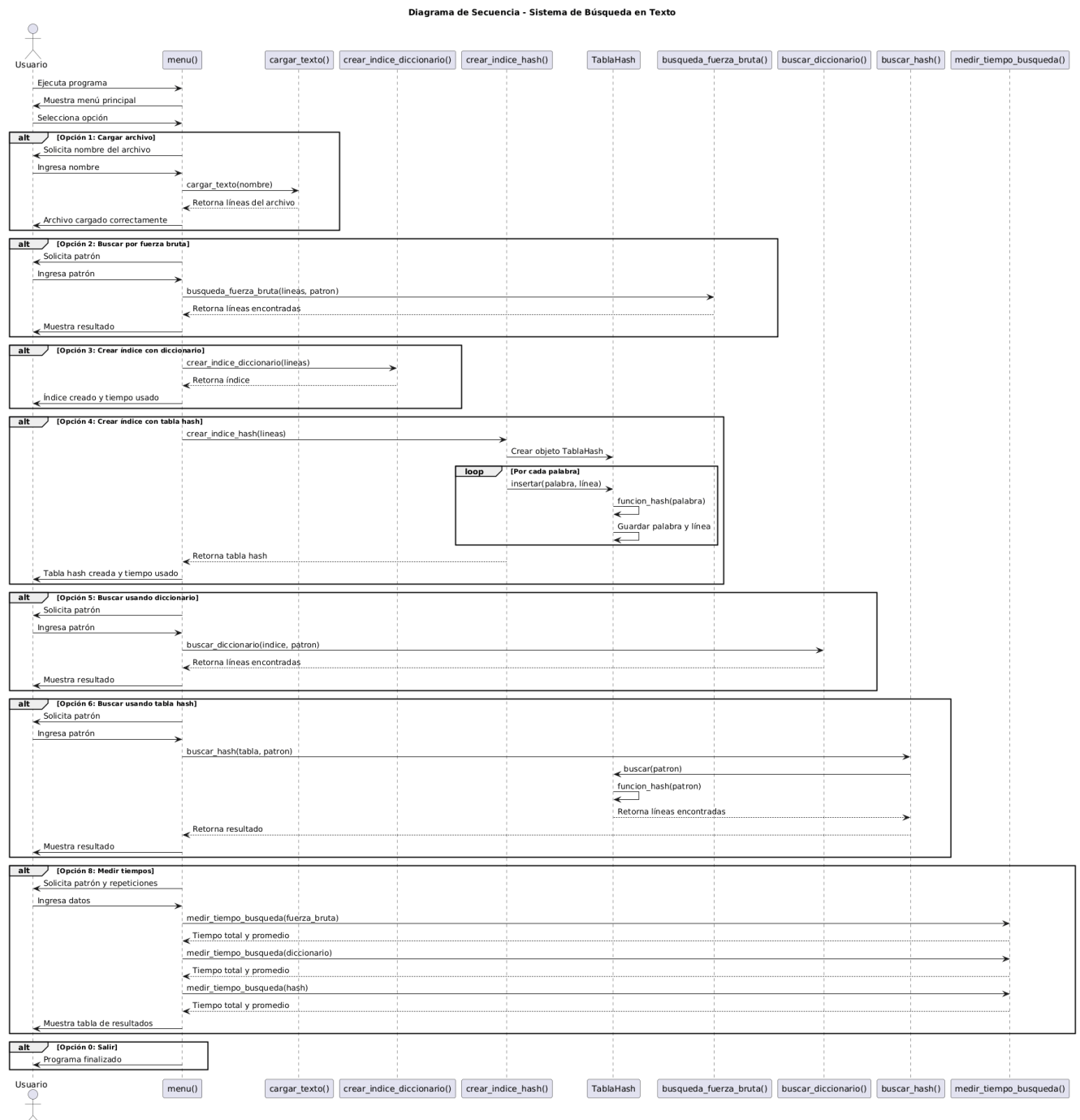


Figura 2: Diagrama de estado de la solución

Implementación

Pseudocódigo de Fuerza Bruta

BuscarFuerzaBruta(líneas, patrón)

Crear lista ocurrencias vacía
Convertir patron a minúsculas

Para cada linea del archivo
Convertir linea a minúsculas

```
'''  
Si patron aparece en linea  
    guardar numero de linea en ocurrencias  
'''
```

Retornar ocurrencias

Pseudocódigo del Índice Hash

CrearIndiceHash(lineas)

Crear tabla hash vacía

Para cada linea del archivo
Separar linea en palabras

```
'''  
Para cada palabra  
    Limpiar palabra  
    Convertir palabra a minúsculas  
  
    Si palabra no está vacía  
        Calcular posición usando función hash  
        Insertar palabra en tabla hash  
        Guardar número de línea  
'''
```

Retornar tabla hash

Cliente Interactivo

El programa cuenta con un menú interactivo que permite al usuario elegir qué acción desea realizar. Las opciones principales son cargar un archivo, buscar una palabra, crear índices y medir tiempos de ejecución.

- **Cargar archivo:** permite ingresar el nombre del archivo de texto que se quiere analizar.
- **Buscar por fuerza bruta:** busca el patrón recorriendo todas las líneas del archivo.
- **Crear índice con diccionario:** crea un índice usando un diccionario de Python.

- **Crear índice con tabla hash propia:** crea un índice usando una tabla hash implementada manualmente.
- **Buscar usando diccionario:** busca el patrón utilizando el índice creado con diccionario.
- **Buscar usando tabla hash:** busca el patrón utilizando la tabla hash propia.
- **Ejecutar consultas desde archivo:** permite leer varios patrones desde un archivo y buscarlos automáticamente.
- **Medir tiempos:** compara el tiempo de búsqueda de fuerza bruta, diccionario y tabla hash.
- **Salir:** finaliza la ejecución del programa.

Modo de Uso

Ejecución

Como el programa está escrito en Python, se puede ejecutar desde la terminal usando el siguiente comando:

```
python3 main.py
```

También puede ejecutarse como:

```
python main.py
```

dependiendo de la instalación de Python del sistema.

Ejemplo de ejecución

```
===== MENU PRINCIPAL =====
```

1. Cargar archivo de texto
2. Buscar por fuerza bruta
3. Crear índice con diccionario de Python
4. Crear índice con tabla hash propia
5. Buscar usando diccionario de Python
6. Buscar usando tabla hash propia
7. Ejecutar consultas desde archivo
8. Medir tiempos
9. Salir

Seleccione una opción: 1

Ingrese nombre del archivo: ElHombreBicentenario.txt

Archivo cargado correctamente.

Cantidad de líneas: 1276

Seleccione una opción: 6

Ingrese patrón a buscar: robot

Líneas encontradas: [42, 44, 46, 57, 62, 72, 74, 75, 93, 95, 96, 153, 163, 186, 203, 2

Pruebas Experimentales

Texto de Prueba 1

```
Seleccione una opción: 1
Ingrese nombre del archivo: ElPrincipito.txt
Archivo cargado correctamente.
Cantidad de líneas: 1580

===== MENU PRINCIPAL =====
1. Cargar archivo de texto
2. Buscar por fuerza bruta
3. Crear índice con diccionario de Python
4. Crear índice con tabla hash propia
5. Buscar usando diccionario de Python
6. Buscar usando tabla hash propia
7. Ejecutar consultas desde archivo
8. Medir tiempos
0. Salir
Seleccione una opción:
```

Figura 3: Prueba con el texto ElPrincipito

Como primer texto de prueba se utiliza el archivo entregado en el enunciado de la tarea, correspondiente a *El Principito*. Este archivo permite comprobar el funcionamiento del programa con un texto conocido y verificar que las búsquedas entreguen las líneas correctas.

Texto de Prueba 2

```
===== MENU PRINCIPAL =====
1. Cargar archivo de texto
2. Buscar por fuerza bruta
3. Crear índice con diccionario de Python
4. Crear índice con tabla hash propia
5. Buscar usando diccionario de Python
6. Buscar usando tabla hash propia
7. Ejecutar consultas desde archivo
8. Medir tiempos
0. Salir
Seleccione una opción: 4
Índice con tabla hash propia creado
Tiempo de creación: 0.024520158767700195 segundos

===== MENU PRINCIPAL =====
1. Cargar archivo de texto
2. Buscar por fuerza bruta
3. Crear índice con diccionario de Python
4. Crear índice con tabla hash propia
5. Buscar usando diccionario de Python
6. Buscar usando tabla hash propia
7. Ejecutar consultas desde archivo
8. Medir tiempos
0. Salir
Seleccione una opción: 6
Ingrese patrón a buscar: robot
Líneas encontradas: [42, 44, 46, 57, 62, 72, 74, 75, 93, 95, 96, 153, 163, 186, 203, 213, 216, 236, 243, 244, 259, 287, 293, 381, 385, 389, 396, 342, 372, 376, 447, 468, 476, 488, 489, 588, 524, 525, 533, 5
6, 593, 595, 597, 599, 602, 603, 604, 626, 628, 658, 788, 716, 721, 726, 734, 735, 739, 740, 741, 753, 773, 776, 783, 784, 785, 798, 851, 906, 933, 934, 1086, 1010, 1072, 1087, 1181, 1191, 1196, 1221, 1250]
```

Figura 4: Pruebas con texto del ElhombreBicentenario

Como segundo texto de prueba se utiliza un archivo de texto obtenido por el grupo, correspondiente a *El Hombre Bicentenario* de Isaac Asimov. Este archivo permite probar el programa con otro contenido y comparar el comportamiento de los métodos de búsqueda.

Metodología

Para realizar las pruebas se cargó cada archivo de texto en el programa y se ejecutaron búsquedas usando tres métodos: fuerza bruta, diccionario de Python y tabla hash propia.

Primero se creó el índice correspondiente para el diccionario y para la tabla hash. Luego se ingresó un patrón de búsqueda y se ejecutó la medición de tiempo varias veces usando la función `time.time()` de Python.

- Se realizaron varias repeticiones para cada método de búsqueda.
- Se utilizó el módulo `time` de Python para medir el tiempo de ejecución.
- Se comparó el tiempo total y el tiempo promedio de cada método.
- Las pruebas se realizaron con los textos *El Principito* y *El Hombre Bicentenario*.
- El tiempo total de experimentación se aproximó a tres minutos, considerando la carga de archivos, creación de índices y medición de consultas.

Resultados

Resultados del Texto 1

```
===== MENU PRINCIPAL =====
1. Cargar archivo de texto
2. Buscar por fuerza bruta
3. Crear índice con diccionario de Python
4. Crear índice con tabla hash propia
5. Buscar usando diccionario de Python
6. Buscar usando tabla hash propia
7. Ejecutar consultas desde archivo
8. Medir tiempos
0. Salir
Seleccione una opción: 1
Ingrese nombre del archivo: ElPrincipito.txt
Archivo cargado correctamente.
Cantidad de líneas: 1580

===== MENU PRINCIPAL =====
1. Cargar archivo de texto
2. Buscar por fuerza bruta
3. Crear índice con diccionario de Python
4. Crear índice con tabla hash propia
5. Buscar usando diccionario de Python
6. Buscar usando tabla hash propia
7. Ejecutar consultas desde archivo
8. Medir tiempos
0. Salir
Seleccione una opción: 8
Ingrese patrón para medir: El
Ingrese cantidad de repeticiones: 4

===== RESULTADOS =====
Método                Tiempo total          Tiempo promedio
Fuerza bruta           0.002953052520751953   0.0007382631301879883
Diccionario Python     5.245208740234375e-06   1.3113021850585938e-06
Tabla Hash propia      5.4836273193359375e-06   1.3709068298339844e-06
```

Figura 5: Tiempo de prueba con el Principito

Resultados del Texto 2

```
Seleccione una opción: 8
Ingrese patrón para medir: robot
Ingrese cantidad de repeticiones: 5

===== RESULTADOS =====
Método                Tiempo total          Tiempo promedio
Fuerza bruta           0.01471400260925293    0.002942800521850586
Diccionario Python     2.4557113647460938e-05  4.911422729492188e-06
Tabla Hash propia      4.100799560546875e-05   8.20159912109375e-06

===== MENU PRINCIPAL =====
1. Cargar archivo de texto
2. Buscar por fuerza bruta
3. Crear índice con diccionario de Python
4. Crear índice con tabla hash propia
5. Buscar usando diccionario de Python
6. Buscar usando tabla hash propia
7. Ejecutar consultas desde archivo
8. Medir tiempos
0. Salir
Seleccione una opción: 8
Ingrese patrón para medir: robot
Ingrese cantidad de repeticiones: 1

===== RESULTADOS =====
Método                Tiempo total          Tiempo promedio
Fuerza bruta           0.003274202346801758    0.003274202346801758
Diccionario Python     1.5497207641601562e-05  1.5497207641601562e-05
Tabla Hash propia      2.0503997802734375e-05  2.0503997802734375e-05
```

Figura 6: Pruebas de tiempo con el hombre del bicentenario

Gráficos

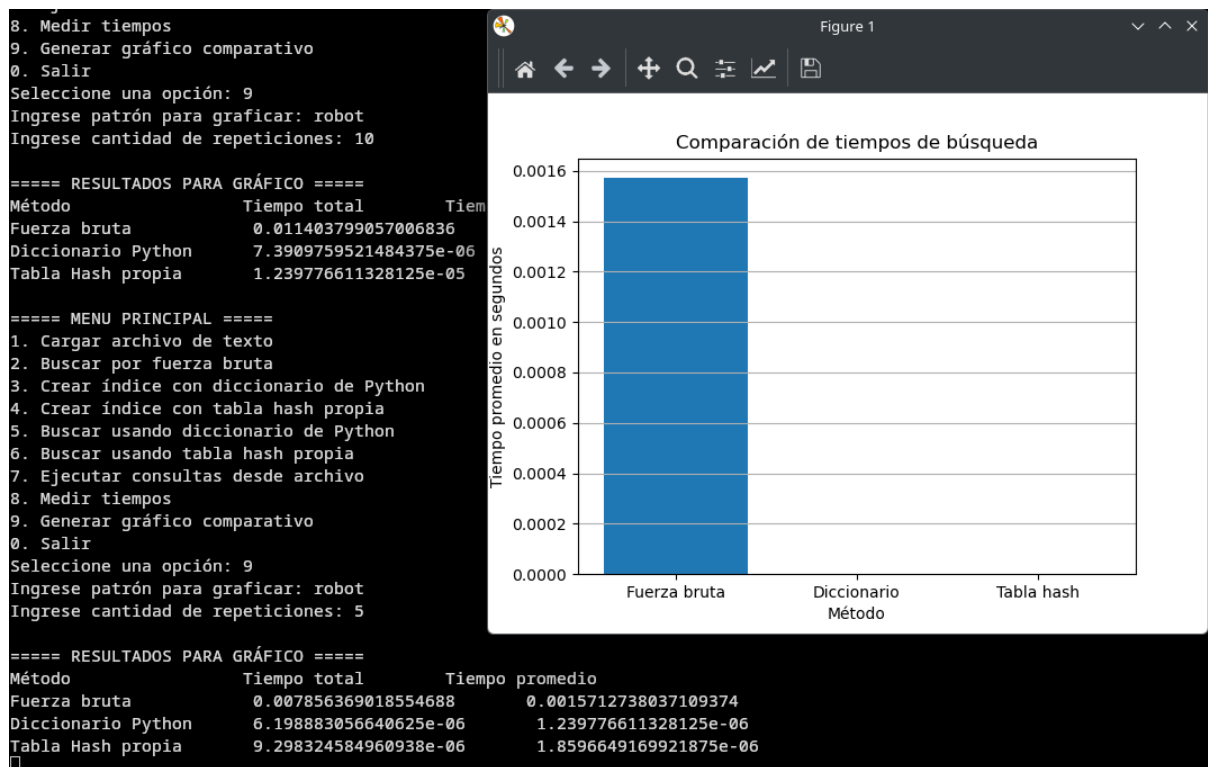


Figura 7: Comparación de tiempos para la palabra robot

Análisis de Resultados

Al comparar los distintos métodos de búsqueda, se observó que la búsqueda por fuerza bruta tarda más tiempo porque debe recorrer todas las líneas del archivo cada vez que se realiza una consulta. En cambio, las búsquedas utilizando diccionario o tabla hash son más rápidas porque utilizan un índice previamente construido.

La creación del índice requiere un tiempo inicial adicional, ya que el programa debe recorrer todo el archivo para almacenar las palabras y las líneas donde aparecen. Sin embargo, este costo se realiza solo una vez.

La principal ventaja del índice aparece cuando se realizan muchas consultas. Después de crear el índice, las búsquedas son mucho más rápidas que con fuerza bruta, ya que no es necesario revisar nuevamente todo el archivo.

Los resultados obtenidos durante las pruebas coinciden con lo esperado según la teoría. La búsqueda secuencial presenta un mayor tiempo de ejecución al aumentar el tamaño del texto, mientras que las búsquedas indexadas mantienen tiempos menores gracias al acceso directo a la información almacenada.

Conclusiones

A partir de las pruebas realizadas se pudo comprobar que las búsquedas indexadas son más eficientes que la búsqueda por fuerza bruta para encontrar información dentro

de archivos de texto.

También se observó que la creación del índice requiere un tiempo inicial de procesamiento, pero este costo se compensa cuando se realizan varias búsquedas sobre el mismo archivo.

La tabla hash y el diccionario permitieron obtener resultados en menos tiempo que la búsqueda secuencial, especialmente al realizar múltiples consultas.

Finalmente, los resultados experimentales obtenidos fueron consistentes con la teoría estudiada, demostrando que las estructuras indexadas mejoran el rendimiento de las búsquedas al reducir la cantidad de datos que deben revisarse en cada consulta.

Anexos

```
import time # Importa el módulo time para medir tiempos

class TablaHash: # Define la clase TablaHash
    def __init__(self, tamano=1000): # Constructor de la tabla hash
        self.tamano = tamano # Guarda el tamaño de la tabla
        self.tabla = [] # Crea la lista principal de la tabla
        for _ in range(tamano): # Repite según el tamaño indicado
            self.tabla.append([]) # Agrega una cubeta vacía

    def funcion_hash(self, palabra): # Calcula el índice hash de una palabra
        suma = 0 # Inicializa la suma
        for letra in palabra: # Recorre cada letra
            suma += ord(letra) # Suma el código ASCII de la letra
        return suma % self.tamano # Retorna un índice válido

    def insertar(self, palabra, linea): # Inserta una palabra y línea
        indice = self.funcion_hash(palabra) # Calcula el índice
        cubeta = self.tabla[indice] # Obtiene la cubeta
        for elemento in cubeta: # Recorre la cubeta
            if elemento[0] == palabra: # Si la palabra ya existe
                if linea not in elemento[1]: # Si la línea no está registrada
                    elemento[1].append(linea) # Agrega la línea
                return # Termina la función
        cubeta.append([palabra, [linea]]) # Agrega una nueva palabra

    def buscar(self, palabra): # Busca una palabra en la tabla
        indice = self.funcion_hash(palabra) # Calcula el índice
        cubeta = self.tabla[indice] # Obtiene la cubeta
        for elemento in cubeta: # Recorre la cubeta
            if elemento[0] == palabra: # Si encuentra la palabra
                return elemento[1] # Retorna las líneas
        return [] # Retorna vacío si no encuentra nada

    def cargar_texto(nombre_archivo): # Carga un archivo de texto
        with open(nombre_archivo, "r", encoding="utf-8") as archivo: # Abre el archivo
```

```
return archivo.readlines() # Retorna sus líneas

def limpiar_palabra(palabra): # Limpia una palabra
    signos = ".,:;!()?[]{}\"'¿;\\n\\t" # Signos a eliminar
    return palabra.strip(signos).lower() # Quita signos y pasa a minúscula

def busqueda_fuerza_bruta(lineas, patron): # Busca recorriendo todas las líneas
    ocurrencias = [] # Lista de líneas encontradas
    patron = patron.lower() # Convierte el patrón a minúscula
    for i in range(len(lineas)): # Recorre las líneas
        linea = lineas[i].lower() # Convierte la línea a minúscula
        if patron in linea: # Verifica si el patrón está en la línea
            ocurrencias.append(i + 1) # Guarda el número de línea
    return ocurrencias # Retorna las ocurrencias

def crear_indice_diccionario(lineas): # Crea índice usando diccionario
    indice = {} # Crea diccionario vacío
    for i in range(len(lineas)): # Recorre las líneas
        palabras = lineas[i].split() # Divide la línea en palabras
        for palabra in palabras: # Recorre cada palabra
            palabra = limpiar_palabra(palabra) # Limpia la palabra

            if palabra != "": # Verifica que no esté vacía
                if palabra not in indice: # Si no existe en el índice
                    indice[palabra] = [] # Crea lista para esa palabra
                if (i + 1) not in indice[palabra]: # Evita repetir línea
                    indice[palabra].append(i + 1) # Agrega número de línea
    return indice # Retorna el índice

def crear_indice_hash(lineas): # Crea índice usando tabla hash propia
    tabla = TablaHash() # Crea una tabla hash
    for i in range(len(lineas)): # Recorre las líneas
        palabras = lineas[i].split() # Divide la línea en palabras

        for palabra in palabras: # Recorre cada palabra
            palabra = limpiar_palabra(palabra) # Limpia la palabra

            if palabra != "": # Verifica que no esté vacía
                tabla.insertar(palabra, i + 1) # Inserta palabra y línea
    return tabla # Retorna la tabla hash

def buscar_diccionario(indice, patron): # Busca usando diccionario
    patron = limpiar_palabra(patron) # Limpia el patrón
    return indice.get(patron, []) # Retorna líneas o lista vacía

def buscar_hash(tabla, patron): # Busca usando tabla hash
    patron = limpiar_palabra(patron) # Limpia el patrón
```



```
return tabla.buscar(patron) # Retorna resultado de la búsqueda

def medir_tiempo_búsqueda(funcion, repeticiones): # Mide tiempo de una búsqueda
    inicio = time.time() # Guarda tiempo inicial
    for _ in range(repeticiones): # Repite la búsqueda
        funcion() # Ejecuta la función recibida

    fin = time.time() # Guarda tiempo final
    tiempo_total = fin - inicio # Calcula tiempo total
    tiempo_promedio = tiempo_total / repeticiones # Calcula promedio
    return tiempo_total, tiempo_promedio # Retorna ambos tiempos

def ejecutar_consultas_archivo(nombre_archivo, lineas, indice_diccionario, indice_hash):
    with open(nombre_archivo, "r", encoding="utf-8") as archivo: # Abre archivo de consultas
        consultas = archivo.readlines() # Lee todas las consultas
    for consulta in consultas: # Recorre cada consulta
        patron = consulta.strip() # Quita espacios y saltos
        if patron != "": # Verifica que no esté vacía
            print("\nPatrón:", patron) # Muestra el patrón
            print("Fuerza bruta:", búsqueda_fuerza_bruta(lineas, patron)) # Busca por fuerza bruta
            print("Diccionario Python:", buscar_diccionario(indice_diccionario, patron)) # Busca en diccionario
            print("Tabla Hash propia:", buscar_hash(indice_hash, patron)) # Busca en tabla hash

def menu(): # Función principal del menú
    lineas = [] # Guarda las líneas del archivo
    indice_diccionario = None # Índice diccionario inicial
    indice_hash = None # Índice hash inicial
    while True: # Ciclo infinito del menú
        print("\n===== MENU PRINCIPAL =====") # Imprime título
        print("1. Cargar archivo de texto") # Opción 1
        print("2. Buscar por fuerza bruta") # Opción 2
        print("3. Crear índice con diccionario de Python") # Opción 3
        print("4. Crear índice con tabla hash propia") # Opción 4
        print("5. Buscar usando diccionario de Python") # Opción 5
        print("6. Buscar usando tabla hash propia") # Opción 6
        print("7. Ejecutar consultas desde archivo") # Opción 7
        print("8. Medir tiempos") # Opción 8
        print("0. Salir") # Opción salir
        opcion = input("Seleccione una opción: ") # Lee opción del usuario

        if opcion == "1": # Si el usuario elige cargar archivo
            nombre = input("Ingrese nombre del archivo: ") # Pide nombre

            try: # Intenta cargar el archivo
                lineas = cargar_texto(nombre) # Carga las líneas
                indice_diccionario = None # Reinicia diccionario
```

```
indice_hash = None # Reinicia hash

print("Archivo cargado correctamente.") # Mensaje de éxito
print("Cantidad de líneas:", len(lineas)) # Muestra cantidad

except FileNotFoundError: # Si no existe el archivo
    print("Error: el archivo no existe.") # Mensaje de error

elif opcion == "2": # Si elige búsqueda fuerza bruta
    if not lineas: # Si no hay archivo cargado
        print("Primero debe cargar un archivo.") # Mensaje de aviso
    else: # Si hay archivo
        patron = input("Ingrese patrón a buscar: ") # Pide patrón
        resultado = busqueda_fuerza_bruta(lineas, patron) # Busca patrón
        print("Líneas encontradas:", resultado) # Muestra resultado

elif opcion == "3": # Si elige crear índice diccionario
    if not lineas: # Si no hay archivo
        print("Primero debe cargar un archivo.") # Mensaje de aviso
    else: # Si hay archivo
        inicio = time.time() # Tiempo inicial
        indice_diccionario = crear_indice_diccionario(lineas) # Crea índice
        fin = time.time() # Tiempo final

        print("Índice con diccionario creado.") # Mensaje de éxito
        print("Tiempo de creación:", fin - inicio, "segundos") # Muestra tiempo

elif opcion == "4": # Si elige crear índice hash
    if not lineas: # Si no hay archivo
        print("Primero debe cargar un archivo.") # Mensaje de aviso
    else: # Si hay archivo
        inicio = time.time() # Tiempo inicial
        indice_hash = crear_indice_hash(lineas) # Crea índice hash
        fin = time.time() # Tiempo final

        print("Índice con tabla hash propia creado.") # Mensaje de éxito
        print("Tiempo de creación:", fin - inicio, "segundos") # Muestra tiempo

elif opcion == "5": # Si elige buscar en diccionario
    if indice_diccionario is None: # Si no existe índice
        print("Primero debe crear el índice con diccionario.") # Mensaje de aviso
    else: # Si existe índice
        patron = input("Ingrese patrón a buscar: ") # Pide patrón
        resultado = buscar_diccionario(indice_diccionario, patron) # Busca patrón
        print("Líneas encontradas:", resultado) # Muestra resultado

elif opcion == "6": # Si elige buscar en hash
```

```
if indice_hash is None: # Si no existe índice hash
    print("Primero debe crear el índice hash.") # Mensaje de aviso
else: # Si existe índice hash
    patron = input("Ingrese patrón a buscar: ") # Pide patrón
    resultado = buscar_hash(indice_hash, patron) # Busca patrón
    print("Líneas encontradas:", resultado) # Muestra resultado

elif opcion == "7": # Si elige ejecutar consultas desde archivo
    if not lineas: # Si no hay archivo cargado
        print("Primero debe cargar un archivo.") # Mensaje de aviso
    else: # Si hay archivo
        if indice_diccionario is None: # Si no existe diccionario
            indice_diccionario = crear_indice_diccionario(lineas) # Lo crea

        if indice_hash is None: # Si no existe tabla hash
            indice_hash = crear_indice_hash(lineas) # La crea

        archivo_consultas = input("Ingrese archivo de consultas: ") # Pide a

    try: # Intenta ejecutar consultas
        ejecutar_consultas_archivo( # Llama función de consultas
            archivo_consultas, # Archivo con patrones
            lineas, # Líneas del texto
            indice_diccionario, # Índice diccionario
            indice_hash # Índice hash
        )
    except FileNotFoundError: # Si el archivo no existe
        print("Error: el archivo de consultas no existe.") # Mensaje de e

elif opcion == "8": # Si elige medir tiempos
    if not lineas: # Si no hay archivo
        print("Primero debe cargar un archivo.") # Mensaje de aviso
    else: # Si hay archivo
        patron = input("Ingrese patrón para medir: ") # Pide patrón
        repeticiones = int(input("Ingrese cantidad de repeticiones: ")) # Pi

        if repeticiones <= 0: # Valida repeticiones
            print("La cantidad de repeticiones debe ser mayor que 0.") # Men
        else: # Si el número es válido
            if indice_diccionario is None: # Si no existe índice diccionario
                indice_diccionario = crear_indice_diccionario(lineas) # Lo c

            if indice_hash is None: # Si no existe índice hash
                indice_hash = crear_indice_hash(lineas) # Lo crea

        total_fb, promedio_fb = medir_tiempo_búsqueda( # Mide fuerza brut
            lambda: busqueda_fuerza_bruta(lineas, patron), # Función anón
```

```
        repeticiones # Cantidad de repeticiones
    )

    total_dic, promedio_dic = medir_tiempo_busqueda( # Mide diccionario
        lambda: buscar_diccionario(indice_diccionario, patron), # Función anónima
        repeticiones # Cantidad de repeticiones
    )

    total_hash, promedio_hash = medir_tiempo_busqueda( # Mide hash
        lambda: buscar_hash(indice_hash, patron), # Función anónima
        repeticiones # Cantidad de repeticiones
    )

    print("\n==== RESULTADOS =====") # Título resultados
    print("Método                Tiempo total                Tiempo promedio")
    print("Fuerza bruta            ", total_fb, "            ", promedio_fb)
    print("Diccionario Python          ", total_dic, "            ", promedio_dic)
    print("Tabla Hash propia            ", total_hash, "            ", promedio_hash)

elif opcion == "0": # Si elige salir
    print("Programa finalizado.") # Mensaje final
    break # Sale del ciclo

else: # Si ingresa opción incorrecta
    print("Opción inválida.") # Mensaje de opción inválida

menu() # Ejecuta el programa
```